



1. Czym jest Haskell?

Haskell jest czysto funkcyjnym, silnie i statycznie typowanym językiem programowania. Program w Haskellu jest zbiorem definicji funkcji, a obliczenie programu polega na wyznaczaniu wartości wyrażeń. W odróżnieniu od języków imperatywnych, takich jak C++, Java czy Python, nacisk nie jest położony na modyfikowanie stanu programu krok po kroku, lecz na opis zależności między danymi wejściowymi i wynikiem.

- Nazwa pochodzi od imienia znanego logika Haskell’a Brooks’a Curry’ego (1900-1982)
- Jest jednym z wielu funkcyjnych języków programowania, do których należą m.in. Lisp, Erlang i inne.
- Jest językiem czysto funkcyjnym, dzięki czemu nie pozwala na efekty uboczne. Głównym założeniem języków czysto funkcyjnych jest to, że wynik działania funkcji jest uzależniony wyłącznie od przekazywanych jej argumentów. Za każdym razem, gdy wywołamy funkcję z takimi samymi parametrami, zwróci ona taka sama wartość.
- Jest językiem „leniwym” (ang. „lazy”, „non strict”), gdyż wyrażenia, które nie są potrzebne, by ustalić odpowiedź na dany problem nie są wyznaczane. Leniwe wartościowanie gwarantuje, że nic nie zostanie policzone niepotrzebnie.
- Jest językiem stosującym silne typowanie. Niemożliwa jest więc przypadkowa (niejawna) konwersja, np. Double do Int.
- Jest zwięzły. Programy są krótsze, dzięki czemu łatwiej jest analizować poszczególne funkcje i lokalizować błędy.
- Jest językiem wysokiego poziomu – programy napisane w Haskellu często są bardzo podobne do opisu algorytmu. Pisząc funkcje na wyższym poziomie abstrakcji i zostawiając szczegóły kompilatorowi, zmniejszamy szanse pojawienia się błędu.
- Zarządza pamięcią – programista zajmuje się jedynie implementacją algorytmu.
- Jest modularny – Haskell oferuje bardzo wiele metod łączenia modułów.
- Język Haskell, podobnie jak języki obiektowe, umożliwia tworzenie abstrakcyjnych struktur danych, polimorfizm i enkapsulację. Enkapsulacja jest realizowana poprzez umieszczenie każdego typu danych w osobnym module.

Idea przewodnia

Funkcja powinna dla tych samych argumentów zwracać zawsze ten sam wynik. Dzięki temu kod łatwiej testować, analizować i przekształcać matematycznie.

Najważniejsze cechy Haskell'a:

- Czystość funkcyjna - funkcje nie zmieniają ukrytego stanu programu; operacje wejścia-wyjścia są oddzielone typem IO.
- Niemutowalność - raz zdefiniowana nazwa nie jest później modyfikowana.
- Leniwa ewaluacja - wyrażenia są obliczane dopiero wtedy, gdy ich wartość jest potrzebna.
- Silne typowanie - kompilator nie wykonuje przypadkowych, niejawnych konwersji typów.
- Wnioskowanie typów - wiele typów może zostać ustalonych automatycznie przez kompilator.
- Zwięzłość - wiele algorytmów można zapisać jako krótkie definicje funkcji.

Lista komend

```
Type :? for help
Hugs> :?
LIST OF COMMANDS: Any command may be abbreviated to :c where
c is the first character in the full name.

:load <filenames>    load modules from specified files
:load                clear all files except prelude
:also <filenames>    read additional modules
:reload              repeat last load command
:edit <filename>      edit file
:edit                edit last module
:module <module>      set module for evaluating expressions
<expr>               evaluate expression
:type <expr>          print type of expression
:?                   display this list of commands
:set <options>        set command line options
:set                 help on command line options
:names [pat]          list names currently in scope
:info <names>          describe named objects
:browse <modules>     browse names exported by <modules>
:main <arguments>     run the main function with the given arguments
:find <name>           edit module containing definition of name
:cd dir               change directory
:gc                   force garbage collection
:version              print Hugs version
:quit                 exit Hugs interpreter
Hugs> |
```

2. Haskell a programowanie imperatywne

Aspekt	Podejście imperatywne	Podejście funkcyjne w Haskellu
Podstawowa jednostka	Instrukcja, procedura, obiekt	Funkcja i wyrażenie
Stan programu	Często modyfikowany przez przypisania	Ograniczony; dane są domyślnie niemutowalne
Pętle	for, while, do-while	Rekurencja, map, filter, fold, listy nieskończone
Efekty uboczne	Naturalna część programu	Kontrolowane jawnie, najczęściej przez typ IO
Typy	Zależnie od języka: statyczne lub dynamiczne	Styczne, silne, z inferencją typów

Przykład pracy w interpreterze

```
2 + 3
10 - 4
5 * 6
20 / 4
:t 5
:t "Haskell"
```

Polecenie `:t` wyświetla typ wyrażenia. Jest to jedna z najważniejszych komend podczas nauki Haskella, ponieważ pozwala sprawdzić, jak kompilator rozumie dane wyrażenie.

```
:t 5
-- 5 :: Num a => a

:t 'a'
-- 'a' :: Char

:t True
-- True :: Bool

:t "Haskell"
-- "Haskell" :: String
```

4. Podstawowe typy danych

Typ	Znaczenie	Przykład
Int	liczba całkowita o ograniczonym zakresie	42
Integer	liczba całkowita dowolnej długości	12345678901234567890
Float	liczba zmiennoprzecinkowa pojedynczej precyzji	3.14
Double	liczba zmiennoprzecinkowa podwójnej precyzji	3.1415926535
Bool	wartość logiczna	True, False
Char	pojedynczy znak	'a'
String	łańcuch znaków, czyli lista znaków [Char]	"Haskell"
[Int]	lista liczb całkowitych	[1,2,3]
(Int, String)	krotka z dwoma elementami różnych typów	(1, "Ala")

5. Operatory, funkcje i priorytety

Operatory arytmetyczne można stosować w postaci infiksowej, natomiast funkcje najczęściej zapisuje się prefiksowo. Haskell pozwala jednak przechodzić między tymi formami.

```
5 + 2
(+) 5 2

5 `div` 2
div 5 2

5 `mod` 2
mod 5 2
```

Operator / funkcja	Znaczenie	Przykład	Wynik
+	dodawanie	3 + 4	7
-	odejmowanie	10 - 6	4
*	mnożenie	5 * 3	15
/	dzielenie rzeczywiste	5 / 2	2.5
div	dzielenie całkowite	5 `div` 2	2
mod	reszta z dzielenia	5 `mod` 2	1
^	potęgowanie całkowite	2 ^ 3	8

Operator	Działanie	Priorytet	Łączność
+	dodawanie	6	L
-	odejmowanie	6	L
*	mnożenie	7	L
/	dzielenie	7	L
^, ^^, **	potęgowanie	8	P
`mod`	dzielenie modulo	7	L
`div`	dzielenie całkowite	7	L
<	mniejszy	4	N
<=	mniejszy lub równy	4	N
>	większy	4	N
>=	większy lub równy	4	N
==	równy	4	N
/=	różny	4	N
&&	AND	3	P
	OR	2	P

Priorytety operatorów decydują o tym, który operator ma pierwszeństwo wykonania przed innym. Priorytet operatora jest liczbą całkowitą z zakresu od 0 do 9 włącznie (im wyższy tym silniejszy). Priorytet 10 jest zarezerwowany dla funkcji.

O kolejności wykonywania operatorów oprócz priorytetu decyduje ponadto własność „fixity”, która określa czy operator wiąże w lewo - L („left-associative” – łączny lewostronnie), w prawo - P („right-associative” – łączny prawostronnie), czy jest niełączny - N („nonassociative”).

Uwaga o priorytetach

Funkcyjne zastosowanie nazwy, np. $\text{mod } 5 \ 2$, ma bardzo wysoki priorytet wiązania. Dlatego wyrażenie $\text{mod } 5 \ 2 \wedge 3$ jest interpretowane jako $(\text{mod } 5 \ 2) \wedge 3$, a nie jako $\text{mod } 5 \ (2 \wedge 3)$.

Wyznaczanie wartości wyrażeń arytmetycznych

Wartości wyrażeń arytmetycznych możemy wyznaczać bezpośrednio w interpreterze.

Uwaga. Funkcje i operatory arytmetyczne mogą być używane zarówno w postaci prefiksowej, jak i infiksowej.

Postać prefiksowa: $\text{div } x \ y$ $\text{mod } x \ y$ $(+) \ 5 \ 2$ $(*) \ 5 \ 2$

Postać infiksowa: $x \ \text{'div'} \ y$ $x \ \text{'mod'} \ y$ $5 + 2$ $5 * 2$

Postać infiksowa jest bardziej naturalna dla operatorów arytmetycznych takich jak $+$ $-$ $*$ $/$ a postać prefiksowa dla funkcji arytmetycznych.

Podstawowe działania

```
Hugs> 1+1
2
Hugs> 8-2
6
Hugs> 11*3
33
Hugs> 35/4
8.75

Hugs> 35 `div` 2
17
```

Wartości i operacje logiczne

```
Hugs> True
True
Hugs> False
False
Hugs> not False
True
Hugs> not True
False
Hugs> 1==1
True
Hugs> 1/=1
False
Hugs> 2<5
True
```

Łańcuchy i znaki

```
Hugs> "To jest string"
"To jest string"
Hugs> 'a'--znak
'a'
```

Nie można łączyć apostrofów z łańcuchami.

łańcuchy można sklejać

```
Hugs> "Hello" ++ " Haskell"
"Hello Haskell"
```

łańcuch jest listą własnych znaków

```
Hugs> ['H', 'e', 'l', 'l', 'o']
"Hello"
```

```
Hugs> "Wyświetlę znak o indeksie 3" !! 3
'w'
```

6. Tworzenie funkcji

Funkcja w Haskellu jest definiowana przez równanie. Zaleca się podawanie jawnej sygnatury typu, zwłaszcza w kodzie dydaktycznym i w sprawozdaniach laboratoryjnych.

```
nazwaFunkcji :: TypArgumentu1 -> TypArgumentu2 -> TypWyniku
nazwaFunkcji argument1 argument2 = wyrażenie
```

Przykłady prostych funkcji

```
dodaj :: Int -> Int -> Int
dodaj a b = a + b

kwadrat :: Float -> Float
kwadrat x = x * x

czyParzysta :: Int -> Bool
czyParzysta x = x `mod` 2 == 0
```

Wywołanie funkcji odbywa się przez podanie nazwy funkcji oraz argumentów oddzielonych spacją. Nawiasy stosuje się głównie do grupowania wyrażeń.

```
dodaj 4 5
kwadrat 3.5
czyParzysta 8

dodaj (kwadratInt 3) 10
  where kwadratInt x = x * x
```

```

1 add a b = a + b
2 -- prosta funkcja przyjmująca 2 argumenty
3
4 doda a b = a+1+b
5 -- inna prosta funkcja
6
7 (//) a b = a `div` b
8 -- nazwa funkcji nie musi zawierać żadnych liter,
9
10 wiek x
11   | x<18 = "Nieletni"
12   | otherwise = "Pełnoletni"
13 -- funkcja z warunkiem
14
15 fun2 1 = 1
16 fun2 2 = 2
17 fun2 x = 10
18 -- funkcja w której Haskell sam wybierze odpowiednią definicję funkcji
19

```

```

Main> add 3 2
5
Main> add 33 11
44

Main> doda 3 3
7

Main> 34 // 2
17

Main> wiek 22
"Pełnoletni"
Main> wiek 13
"Nieletni"

Main> fun2 3
10

```

Funkcje możemy pisać również o określeniem typów

```

1 mnoz :: Int -> Int -> Int
2 mnoz x y = x * y
3
4 mnoz2 :: Int -> Int -> Int
5 mnoz2 x y = if x==0 || y == 0 then 0 else x * y
6
7 mnoz3 :: Int -> Int -> Int
8 mnoz3 x y = if x==0 then 0 else x*y

```

8. Warunki: if, strażniki i case

8.1. Wyrażenie if

W Haskellu if jest wyrażeniem, a nie instrukcją. Musi posiadać zarówno gałąź then, jak i gałąź else, ponieważ całe wyrażenie zwraca wartość.

```
wartoscBezWzgledna :: Int -> Int
wartoscBezWzgledna x =
    if x < 0 then -x else x
```

```
1 mnoz :: Int -> Int -> Int
2 mnoz x y = x * y
3
4 mnoz2 :: Int -> Int -> Int
5 mnoz2 x y = if x==0 || y == 0 then 0 else x * y
6
7 mnoz3 :: Int -> Int -> Int
8 mnoz3 x y = if x==0 then 0 else x*y
```

8.2. Strażniki

Strażniki są wygodne, gdy funkcja ma kilka warunków. Warunki są sprawdzane od góry do dołu.

```
klasyfikujWiek :: Int -> String
klasyfikujWiek wiek
| wiek < 0    = "błędny wiek"
| wiek < 13   = "dziecko"
| wiek < 18   = "młodzież"
| wiek < 65   = "dorosły"
| otherwise  = "senior"
```

```
39 --instrukcja warunkowa IF - działanie na przykładzie funkcji
40 --która zwraca 10 dla wartości 1 i 20 dla każdej innej
41 --napisana w dwóch formach
42
43 warunek1 :: Int -> Int
44 warunek1 x
45     | x==1 = 10
46     | otherwise = 20
47
48 warunek2 :: Int -> Int
49 warunek2 x = if x==1 then 10 else 20
```

8.3. Konstrukcja case

case służy do wyboru wyniku na podstawie dopasowania wzorca. W prostych przykładach może zastępować rozbudowane if, a w bardziej zaawansowanych programach jest podstawą pracy z typami algebraicznymi.


```

dzienTygodnia :: Int -> String
dzienTygodnia n =
  case n of
    1 -> "poniedziałek"
    2 -> "wtorek"
    3 -> "środa"
    4 -> "czwartek"
    5 -> "piątek"
    6 -> "sobota"
    7 -> "niedziela"
    _ -> "nie ma takiego dnia"

```

```

51 podziel :: (Float, Float) -> Float
52 podziel (a,b) =
53     case (a,b) of
54         (0,0) -> 0
55         (_,1) -> a
56         (_,0) -> error "dzielenie przez 0"
57         (a,b) -> a/b
58
59 resztaz :: (Int, Int) -> Int
60 resztaz (a,b) =
61     case (a,b) of
62         (0,0) -> error "dzielenie przez 0"
63         (_,0) -> a `div` 0
64         (0,_) -> 0
65         (a,b) -> a `div` b

```

9. Łańcuchy znaków, listy i zakresy

String w Haskellu jest listą znaków. Oznacza to, że wiele operacji listowych można stosować również do napisów.

```

"Programowanie " ++ "funkcyjne"
reverse "Haskell"
"Informatyka" !! 3

[1..10]
[5,10..30]
[10,9..1]
['a'..'e'] ++ ['f'..'j']

```

Operacja	Znaczenie	Przykład
++	łączenie list lub napisów	"A" ++ "B"
:	dodanie elementu na początek listy	0 : [1..5]
!!	pobranie elementu o indeksie	[1..10] !! 2
head	pierwszy element listy	head [1..4]
tail	lista bez pierwszego elementu	tail [1..4]
last	ostatni element listy	last [1..4]
init	lista bez ostatniego elementu	init [1..4]

Indeksowanie

Listy i napisy są indeksowane od zera. Wyrażenie "Informatyka" !! 3 zwraca czwarty znak, czyli o indeksie 3.

Operacje na listach:

```
Hugs> head [1..4]
1
Hugs> tail [1..4]
[2,3,4]
Hugs> init [1..4]
[1,2,3]
Hugs> [1..4]
[1,2,3,4]
Hugs> last [1..4]
4
```

Listy oraz krotki

Wszystkie elementy listy **muszą być tego samego typu**.

Poniższe dwie listy są identyczne:

```
Hugs> [1,2,3,4,5] == [1..5]
True
```

Zakresy są uniwersalne.

```
Hugs> ['A'..'F']
"ABCDEF"
```

Przy tworzeniu zakresów **można określić krok**

```
Hugs> [0,2..10]
[0,2,4,6,8,10]
```

Zakresy w Haskellu tworzone są **domyślnie rosnąco**

```
Hugs> [5..1]
[]
```

Chyba, że określimy krok

```
Hugs> [5,4..1]
[5,4,3,2,1]
```

Listy jak i stringi **indeksowane są od zera**

```
Hugs> [1..10] !! 2
3
```

Można nawet tworzyć listy nieskończone

```
Hugs> [1..]
```

Nieskończone listy mają prawo działać, ponieważ Haskell cechuje się leniwym wartościowaniem. To oznacza, że obliczane są jedynie te elementy listy, których istotnie potrzebujemy. Możemy poprosić o tysięczny element i dostaniemy go:

```
Hugs> [1..] !! 1000
1001
```

Haskell wyznaczył pierwsze tysiąc elementów listy, ale cała jej reszta jeszcze nie istnieje! Nie zostanie obliczona ich wartość, póki nie zajdzie taka potrzeba.

łączenie dwóch list

```
Hugs> [1..5] ++ [6..10]
[1,2,3,4,5,6,7,8,9,10]
```

dodawanie pojedynczego elementu na początek listy

```
Hugs> 0:[1..5]
[0,1,2,3,4,5]
```

10. Pętle w Haskellu: rekurencja, map, filter i fold

Haskell nie używa klasycznych pętli for i while w takim sensie, jak języki imperatywne. Powtarzanie obliczeń zapisuje się najczęściej przez rekurencję albo funkcje wyższego rzędu.

10.1. Rekurencja

```
silnia :: Int -> Int
silnia 0 = 1
silnia n = n * silnia (n - 1)

sumaDo :: Int -> Int
sumaDo 0 = 0
sumaDo n = n + sumaDo (n - 1)
```

10.2. Funkcje map, filter i fold

map przekształca każdy element listy, filter wybiera elementy spełniające warunek, a foldr lub foldl redukuje listę do jednej wartości.

```
map (+3) [1,2,3,4,5]
filter even [1..10]
foldr (+) 0 [1..5]
```

Funkcja	Opis	Przykład wyniku
map	zastosowanie funkcji do każdego elementu listy	map (*2) [1,2,3] = [2,4,6]
filter	wybranie elementów spełniających predykat	filter even [1..6] = [2,4,6]
foldr / foldl	złożenie listy do jednej wartości	foldr (+) 0 [1..4] = 10

11. Funkcje anonimowe lambda

Funkcja anonimowa nie ma własnej nazwy. W Haskellu zapisuje się ją z użyciem znaku \. Lambdy są użyteczne wtedy, gdy niewielka funkcja jest potrzebna tylko lokalnie.

```
map (\x -> x + 3) [1,2,3,4,5]
filter (\x -> x `mod` 3 == 0) [1..20]
map (\tekst -> reverse tekst) ["Ala", "Haskell", "kod"]
```

12. Leniwa ewaluacja i listy nieskończone

Leniwa ewaluacja oznacza, że Haskell oblicza tylko te fragmenty wyrażenia, które są potrzebne do uzyskania wyniku. Dzięki temu można pracować z listami potencjalnie nieskończonymi, o ile pobieramy z nich skończoną część.

```
liczbyNaturalne :: [Integer]
liczbyNaturalne = [1..]

parzyste :: [Integer]
parzyste = [2,4..]

pierwszeDziesiecParzystych :: [Integer]
pierwszeDziesiecParzystych = take 10 parzyste

piecdziesiatyElement :: Integer
piecdziesiatyElement = parzyste !! 49
```

Dlaczego indeks 49?

Indeksy list zaczynają się od zera. Pierwszy element ma indeks 0, więc pięćdziesiąty element ma indeks 49.

13. Komentarze i styl kodu

Czytelność jest szczególnie ważna w programowaniu funkcyjnym, ponieważ zwężony kod może być dla początkujących trudniejszy do analizy. Każda funkcja w sprawozdaniu powinna posiadać sygnaturę typu, krótką nazwę, komentarz oraz przykładowe wywołanie.

```
-- Komentarz jednoliniowy

{-
  Komentarz wieloliniowy.
  Może obejmować kilka linii.
-}

poleKola :: Double -> Double
poleKola r = pi * r ^ 2
```

14. Minimalny program w pliku .hs

Funkcje można testować w GHCi bez funkcji main, ale kompletny program wykonywalny powinien zawierać funkcję wejściową main o typie IO ().

```
main :: IO ()
main = putStrLn "Witaj w Haskellu!"

-- Uruchomienie w GHCi:
-- :load plik.hs
-- main
```